Developer Guide

Integrating the design engine into web, backend, and admin systems

LA Peptides Universal Label System

Developer Guide

For the web team, backend developers, and anyone integrating this system into another application (website, sales app, admin portal, future APIs). This package has **no dependency on Claude, any AI tool, this repository, or any specific computer** — everything needed to build labels is inside this folder.

Installation

```
cd LA-Peptides-Universal-Label-System
npm install
```

Node.js 18+ only. `npm install` pulls public npm packages (`qrcode` , `sharp` , `pdfkit` , `svg-to-pdfkit` , `pdf-lib` , `archiver` , `@xmldom/xmldom` , `xpath`) — no private registries, no vendored binaries, no network calls at build time beyond the initial install.

Folder structure

```
LA-Peptides-Universal-Label-System/
├── master-label.svg              canonical visual source
├── label-spec.json               field/theme registry (element ids, applyAs,
required, maxLength)
├── layout.json                   percentage-based region geometry + lock/
placeholder/autofit metadata
├── manifest.json                 package table of contents — read this first
├── default-theme.json            neutral starter palette
├── example-brand-theme.json      second palette (legacy theme-file interface,
still supported)
├── brand-profiles/               brand partner identity + styling (recommended
authoring format)
├── product-profiles/             per-product/batch content (recommended authoring
format)
├── placeholder-logo.svg
├── placeholder-qr.svg
├── placeholder-icons.svg
├── master-label-preview.png      committed master deliverable (default theme, all
placeholders)
├── master-label-print.pdf        committed master deliverable
├── master-label-editable.pdf     committed master deliverable
├── schema/                       JSON Schemas for every JSON file in this package
├── scripts/                      build pipeline (see below)
├── README.md, QUICK-START.md, RENDERING-SPEC.md, VALIDATION-RULES.md,
│   DEVELOPER-GUIDE.md (this file), PRODUCTION-GUIDE.md, BRAND-PARTNER-GUIDE.md
├── LICENSE.md, CHANGELOG.md, VERSION
```

File naming standard: `<name>.brand-profile.json`, `<name>.product-profile.json`, `<name>.theme.json` (legacy interface) — the suffix always identifies the schema it validates against, so any file's purpose is obvious from its name alone without opening it.

Loading the master template

You almost never touch `master-label.svg` directly. The integration surface is three JSON files:

1. **`label-spec.json`** — read once. Tells you every element id, whether it's field-mappable (`fields[]`), and how theme tokens map to attributes (`themeTargets`).
2. **`layout.json`** — read once. Gives you every region's position as a **percentage** (0–100) of the canvas, so you can position an HTML overlay, a WYSIWYG editor's selection box, or recompute pixel coordinates at any render resolution — without ever parsing the SVG.

3. `manifest.json` — the discovery entry point. `editableRegionIds`, `contentFieldIds`, and `requiredContentFieldIds` are the three lists most integrations need first.

Replacing every asset type

All of the following are **fields** — set them in a `product-profile.json` / `brand-profile.json` pair (recommended) or a flat field-map (`{"brand_logo": "...", ...}`, legacy interface, still supported by `scripts/build.js --fields`). See `schema/brand-profile.schema.json` / `schema/product-profile.schema.json` for the authoritative property list; every property's `description` states its `-> fieldKey` target.

Replace...	How
Logo	`brand-profile.logo.primary` — path to an `.svg` / `.png` / `.jpg`. Auto-scaled to fit its box, aspect ratio preserved, never distorted (see `VALIDATION-RULES.md`).
Secondary logo / badge	`brand-profile.logo.secondary` — a symbol reference (`"#icon-logo-secondary"`, the built-in seal) or a swapped-in asset. `null` hides it.
Colors	`brand-profile.colors.{background,primary,secondary,accent,border,iconColor}` — 6-digit hex only.
Fonts	`brand-profile.fonts.{heading,subheading,body,mono}` — full CSS font-family stacks, not bare names.
Icons (purity/lab-tested)	`brand-profile.icons.{purity,labTested}` — a symbol reference or a swapped-in asset.
Country/origin graphic	`brand-profile.countryGraphic`.
Decorative accent graphic	`brand-profile.decorativeGraphic`.
QR code	Never set directly — set `product-profile.coaQrDestination` (a URL) and a real, scannable QR is generated automatically at build time.
QR caption text	`brand-profile.qrStyle.caption` (brand default) or `product-profile.qrCaption` (per-product override).
Text (name, tagline, strength, descriptor, LOT, UBD, COA URL, ...)	The corresponding `brand-profile` / `product-profile` string property — see the schema files for the full list.

Building

```
# Recommended: brand-profile.json + product-profile.json
node scripts/build-profile.js --brand brand-profiles/your-partner.brand-
profile.json \
                                --product product-profiles/your-product.product-
profile.json \
                                --out .build/your-product --zip [--strict]

# Legacy: theme.json + flat field-map.json (still supported, same output)
node scripts/build.js --theme your-theme.json --fields your-fields.json --
out .build/your-build --zip
```

Both entry points call the same `buildLabel()` core (`scripts/build.js`) — a brand-profile build and an equivalent theme+field-map build produce byte-identical output. See `RENDERING-SPEC.md` for the exact algorithm, and `scripts/lib/composeProfiles.js` for the profile → theme/field-map mapping.

`--strict` makes validation warnings (not just errors) fail the build — see `VALIDATION-RULES.md`.

Embedding the pipeline instead of shelling out

```
const { buildFromProfiles } = require('./scripts/build-profile');
await buildFromProfiles({ brand: 'brand-profiles/x.json', product: 'product-
profiles/y.json', out: '.build/x', zip: false, strict: false });
```

Or reuse the individual modules in `scripts/lib/` (`applyTheme`, `applyFields`, `autoFit`, `validateProfiles`, `generatePng`, `generatePdf`, `verifyPdf`) directly if you're building a different orchestration (e.g. a queue-based backend job).

Exporting a production-ready PDF for the Epson ColorWorks C6000

1. Run a build (above) — `label-print.pdf` in the output directory is already an exact-physical-size (1.77in × 0.77in / 127.44pt × 55.44pt) vector PDF, dimension-verified against `label-spec.json` at build time (the build fails if the PDF isn't exactly that size).
2. Open it, or send it directly to the Epson driver.
3. Print dialog: **Media size** matching your label stock, **Scale = 100% / Actual Size** (never "Fit to page"), color management left to the driver (sRGB in, printer profile out) unless your vendor supplied a specific ICC profile.
4. No bleed/crop marks are needed or present — this format is a single pre-sized die-cut/kiss-cut label; the print is the trim.

Full non-technical print-review checklist: `PRODUCTION-GUIDE.md`.

Editing in Adobe Acrobat

`master-label-editable.pdf` (and any build's `label-print.pdf`) is a real vector PDF — text and shapes are live objects, not a flattened image. Acrobat's "Edit PDF" tool can adjust text content and basic properties directly. For anything beyond minor text tweaks (recoloring, repositioning, swapping a logo), edit the source `label.svg` instead and rebuild — Acrobat is a review/minor-edit tool here, not the source of truth.

Editing in Adobe Illustrator

`master-label.svg` (or any build's `label.svg`) opens natively in Illustrator with every element as a fully editable, named vector object (Illustrator shows SVG element ids in the Layers panel). This is the right tool for structural changes — moving/resizing a region, redrawing an icon, adjusting the master layout. After a structural edit, update `layout.json`'s corresponding region geometry to match (see `RENDERING-SPEC.md`) — the JSON and SVG must stay in sync since downstream systems read positions from `layout.json`, not by re-parsing the SVG.

Editing in Canva

Canva can import SVG as a design element. Full fidelity for text/shape editing is good; Canva does not preserve SVG `<symbol>` / `<use>` icon references on import (Canva flattens these to static shapes), so icon recoloring via theme tokens won't survive a Canva round-trip — treat a Canva-imported copy as a one-way visual reference/mockup, not something you re-export back into this pipeline.

Versioning strategy

Semantic versioning (`VERSION`, `CHANGELOG.md`, `label-spec.json` → `templateVersion`, `layout.json` → `templateVersion`, `manifest.json` → `version`, `master-label.svg` → `data-template-version` attribute — all five are kept in lockstep). A breaking change to element ids, schema shapes, or physical dimensions is a major version bump. Adding new optional fields/tokens without breaking existing profiles is a minor bump. Pin to a specific version/tag if you're building this into automated infrastructure that can't tolerate an unannounced breaking change.